

TP_PrograB

Estado del trabajo y plan de prioridades

Qué conservar, qué corregir y qué desarrollar para cumplir la consigna.

Evaluación: 13 de septiembre de 2026. Referencia de código: commit 47e0df9, que incorpora cuartos, semifinales, final y penales.

Este plan no modifica el proyecto. Distingue implementación encontrada por inspección de funcionalidad validada mediante pruebas. Las eliminatorias nuevas requieren pruebas específicas.

Cómo leer las prioridades

- P0 - Crítica: corregir primero. Protege resultados, continuidad del torneo y participación válida de jugadores.
- P1 - Alta: completar reglas obligatorias y verificar clasificación y ganadores antes de expandir funcionalidades.
- P2 - Media: incorporar estadios, ABM y estadísticas sobre una base ya confiable. Sigue siendo obligatorio para la entrega.
- P3 - Integración y cierre: interfaz funcional, reportes y entregables. No significa opcional; significa que depende de la lógica anterior.

Orden recomendado

1. Continuidad del torneo guardado.
2. Suspensiones y coherencia de eventos.
3. Desempates, penales y criterio del ganador.
4. Estadios y ABM de ciudades/estadios.
5. Estadísticas confiables.
6. Interfaz y reportes.
7. Validación integral y entrega.

Decisión acordada: JavaFX queda separado por ahora. La terminal es una herramienta temporal de prueba; no es necesario desarrollar otra presentación completa de eliminatorias si se reemplazará por JavaFX.

1. Lo ya implementado: conservar y verificar

Estas tareas no deben rehacerse. Deben conservarse y cubrirse con pruebas de regresión.

Datos y carga

Lectura desde JSON; datos personales de jugadores, DT y árbitros; equipos con país, ranking y DT; valoración y posición; títulos del DT y años de referato.

Planteles y sorteo

Validación de 16 equipos y planteles de 18 jugadores en distribución 2/6/5/5. Sorteo de cuatro zonas equilibradas mediante bombos de ranking.

Fase de grupos

24 encuentros únicos, tres jornadas de ocho partidos y puntuación 3/1/0. Tabla con PTS, PJ, PG, PE, PP, GF, GC y DG.

Simulación y alineaciones

Resultado multifactorial con aleatoriedad; árbitros elegibles por nacionalidad; elección de formaciones compatibles y conservación de titulares.

Incidencias

Registro de goles, minuto, autor, penal/en contra y arquero; cambios y expulsiones. Existe la sanción de una fecha, pero su cumplimiento efectivo necesita la corrección C2.

Eliminatorias

Clasificados obtenidos de las tablas; cruces de cuartos y semis según la consigna; ida y vuelta y final única. Resolución por puntos, goles visitantes dobles y penales.

Penales y persistencia

Penales con pateador y convertido/fallado, tanda inicial y muerte súbita. Serialización de Championship y reparación de jornadas antiguas; continuidad de eliminatorias aún incompleta.

Base de datos e interfaz

Lectura JDBC de ciudades y estadios y relación Stadium -> City -> Country. Menú JavaFX con estilos y animaciones, todavía sin integración funcional.

Diseño orientado a objetos

Uso de encapsulamiento, herencia, colecciones polimórficas, excepciones y separación inicial entre carga, simulación, persistencia y presentación.

Atención: la clasificación de los dos primeros está implementada, pero puede ser incorrecta cuando se llega al último desempate de grupos. Corregir C4 antes de darla por cerrada.

2. Cambiar lo realizado: prioridad P0

C1. Continuidad de eliminatorias guardadas

CORREGIR | P0 - PRIMERO

El main vuelve a ejecutar cuartos, semifinales y final al cargar un torneo. Guardar fase actual, series existentes, clasificados, ganadores y campeón; simular solo lo pendiente. Guardar también al finalizar cada etapa o paso recuperable.

Se considera terminado cuando: Salir y volver a cargar no agrega encuentros duplicados, no cambia resultados ya jugados ni vuelve a elegir otro campeón. Un torneo normal contiene 37 partidos: 24 + 8 + 4 + 1.

Área de trabajo: Core/Run/Championship, Core/persistence/ChampionshipRepository y estado de las series.

C2. Suspensión efectiva durante todo el partido

CORREGIR | P0 - SEGUNDO

Un suspendido se excluye del once, pero al limpiarse su sanción puede reaparecer en el banco construido desde el plantel completo. Preparar una lista de jugadores habilitados para el encuentro y usarla también en los cambios.

Se considera terminado cuando: Un expulsado no aparece ni de titular ni como sustituto en su siguiente partido; después vuelve a estar disponible. Las consultas de formaciones no consumen sanciones.

Área de trabajo: Core/domain/Player, Lineup y Core/simulation/MatchSimulator.

C3. Coherencia cronológica y jugadores en cancha

CORREGIR | P0 - TERCERO

Actualmente los grupos de eventos se generan por separado. Controlar el estado de jugadores en cancha, banco y expulsados en función del minuto. No es obligatorio implementar un reloj en tiempo real: sí un estado cronológico consistente.

Se considera terminado cuando: Nadie marca después de salir o ser expulsado; los cambios solo involucran jugadores válidos y no permiten regresar a un sustituto. Las formaciones iniciales se conservan como registro histórico.

Área de trabajo: MatchSimulator, Lineup y clases Incident.

Pruebas P0

- Guardar/cargar tras grupos, ida, vuelta, semifinal y final; verificar resultados y cantidad de encuentros.
- Forzar una expulsión y comprobar la ausencia completa del jugador en el encuentro siguiente.
- Revisar líneas temporales con expulsiones y cambios antes y después de goles.

No avanzar a estadísticas de minutos ni selección de pateadores finales sin resolver C2 y C3: esas funcionalidades dependen de saber quién participó realmente.

3. Cambiar lo realizado: prioridad P1

C4. Desempate reglamentario de grupos

CORREGIR | P1 - ALTA

Reemplazar el ranking como último criterio obligatorio por el resultado entre ambos equipos empatados. Mantener el orden: puntos, diferencia de gol, goles a favor, resultado entre ambos. Si queda una igualdad que la consigna no resuelve, consultar a la cátedra.

Se considera terminado cuando: Una tabla de prueba con igualdad en PTS, DG y GF respeta el encuentro directo. Los clasificados cambian solo conforme al reglamento.

Área de trabajo: Championship.getStandings y pruebas de clasificación.

C5. Conservar y exponer el criterio del ganador

COMPLETAR | P1 - ALTA

Registrar el ganador y si avanzó por puntos, diferencia ponderada o penales; en final, por resultado de los 90 minutos o penales. Utilizar settledByPenalties y conservar el resultado de la tanda.

Se considera terminado cuando: Se puede consultar el criterio después de cargar el torneo y mostrarlo en cualquier interfaz sin recalcular ni repetir penales.

Área de trabajo: SecondLegMatch, FinalMatch, resolución de series y modelo de resultado.

C6. Pateadores habilitados para la tanda

CORREGIR | P1 - ALTA | DEPENDE DE C2/C3

La tanda toma el once inicial. Usar participantes habilitados al finalizar el encuentro, considerando sustituciones y expulsiones. No es un error por sí mismo que un arquero pateee: lo importante es la elegibilidad.

Se considera terminado cuando: No patea ningún expulsado ni sustituido; se conserva cada ejecución con pateador y convertido/fallado.

Área de trabajo: determinePenaltyShootoutWinner, simulatePenalty y PenaltyShootout.

C7. Pruebas de reglas eliminatorias

DESARROLLAR PRUEBAS | P1 - ALTA

Agregar pruebas de puntos en ida/vuelta, diferencia con visitante doble, penales, final empatada, cruces exactos y persistencia sin duplicación. Las pruebas se desarrollan junto con C1-C6, no solo al final.

Se considera terminado cuando: Casos controlados y reproducibles verifican cada camino de decisión y no requieren una interfaz.

Área de trabajo: src/test/java.

Regla ya correcta: no sustituir los puntos y goles visitantes dobles por reglas de fútbol actuales. El reglamento particular del TP exige precisamente esos criterios.

4. Desarrollar: estadios y ABM - P2

N1. Asociar y sortear estadios eliminatorios

NUEVO | P2 - OBLIGATORIA

Agregar el estadio al partido. Elegir aleatoriamente entre los cargados de PostgreSQL y retirar el usado del conjunto disponible. Conservar los estadios usados dentro del estado serializado.

Se considera terminado cuando: Los 13 encuentros eliminatorios usan 13 estadios distintos, también después de salir y cargar. La falta de estadios suficientes produce una validación clara antes de comenzar.

Área de trabajo: Match, Championship, Stadium y carga de sedes.

N2. Alta, baja y modificación de ciudades

NUEVO | P2 - OBLIGATORIA

Añadir operaciones JDBC de alta, baja y modificación con validación de datos y referencias. No hace falta conectar JavaFX todavía: puede probarse el repositorio aisladamente.

Se considera terminado cuando: Las operaciones persisten y la base rechaza eliminar una ciudad con estadios. Verificar la FK real: esa restricción no fue comprobada directamente en pgAdmin.

Área de trabajo: Infrastructure/database/CityRepository y PostgreSQL.

N3. Alta, baja y modificación de estadios

NUEVO | P2 - OBLIGATORIA

Añadir operaciones JDBC y asegurar que cada estadio pertenece a una ciudad existente. Mantener una política clara si se intenta modificar o borrar una sede que ya forma parte de un torneo guardado.

Se considera terminado cuando: Alta/modificación/baja funcionan y una ciudad inexistente no puede asignarse. Se prueban integridad referencial y errores controlados.

Área de trabajo: StadiumRepository, validaciones de dominio y PostgreSQL.

Dependencias y trabajo paralelo

- N2 y N3 pueden desarrollarse en paralelo con estadísticas, una vez estabilizadas las interfaces del dominio.
- N1 puede usar la lectura JDBC ya existente, pero debe completarse antes de considerar terminadas las eliminatorias.
- No confundir ABM con la carga inicial: que ya exista findAll no elimina la obligación de implementar alta, baja y modificación.

La selección de estadios no corresponde a CSS ni a FXML. Primero resolver y probar la regla en el núcleo; luego una pantalla puede mostrar o administrar sus datos.

5. Desarrollar: estadísticas y reportes

N4. Estadísticas propias del campeonato

NUEVO | P2 - BASE DE LOS REPORTES

Calcular partidos y minutos por jugador, goles normales y de penal, goles recibidos por arquero, tarjetas, resultados por equipo y partidos por árbitro. Separar estas estadísticas de las históricas cargadas en Player.statistics.

Se considera terminado cuando: Los valores se obtienen de encuentros e incidencias coherentes y no se duplican al cargar o consultar varias veces.

Área de trabajo: Servicios de estadísticas y modelo de participación; depende de C2/C3.

Reportes obligatorios - P3, después de sus datos base

- N5. Goleadores: goles de partidos y cuántos fueron de penal; excluir las tandas de definición.
- N6. Fair Play por equipos: acordar/documentar el puntaje utilizado y calcularlo desde sanciones del torneo.
- N7. Minutos y partidos: ranking de jugadores según participación real.
- N8. Cuadro de cruces: clasificados por zona, cuartos, semis, final y ganadores. Depende del estado persistente de C1/C5.
- N9. Equipos alfabéticos: edad promedio de jugadores, edad/nacionalidad del DT, GF/GC y efectividad sobre puntos posibles. Definir el tratamiento de tandas y final según la catedral.
- N10. Árbitros: ranking por partidos dirigidos, años de referato y promedio de estos años al final.
- N11. Jugadores por posición: Todos o uno particular; datos completos, partidos, minutos y goles. Arqueros: recibidos y promedio por partido.
- N12. Identificaciones PDF: datos personales, rol, fotografía y código de barras de todas las personas participantes.

Orden para estos reportes

Primero N4. Luego N5/N6/N7/N9/N10/N11, que reutilizan estadísticas. N8 depende del estado de competencia y puede desarrollarse en paralelo. N12 requiere fotografías, un identificador y generación de códigos de barras, pero no depende de los minutos jugados.

Se considera terminado un reporte cuando puede emitirse en cualquier momento del torneo, contempla valores todavía no acumulados y sigue siendo correcto después de cargar una partida.

6. Presentación, calidad y entrega - P3

C8. Tabla antes y después de cada resultado

COMPLETAR PRESENTACIÓN | P3 | REQUISITO OBLIGATORIO

Hoy se muestra después de cada jornada. La entrega debe permitir ver el estado de la tabla antes y después de cada resultado registrado. Resolverlo en JavaFX cuando se conecte la interfaz; no duplicar necesariamente la solución en terminal.

Se considera terminado cuando: El usuario puede identificar ambos estados por partido y consultar todas las columnas exigidas.

Área de trabajo: Presentación JavaFX y coordinación de simulación.

N13. Conectar la interfaz final

NUEVO | P3 | POR AHORA POSPUESTO

Crear/cargar torneo, avanzar por fases, consultar partidos y eventos, mostrar tablas, criterios del ganador, cuadro de cruces, reportes y ABM. Conservar el diseño visual y ejecutar operaciones a través de la lógica, no SQL en FXML.

Se considera terminado cuando: El flujo completo es operable desde JavaFX y no necesita la terminal para funciones obligatorias.

Área de trabajo: Controller, GUI, recursos FXML/CSS y capa de coordinación.

C9. Reducir responsabilidades de Championship

MEJORA DE DISEÑO | TRANSVERSAL

Separar resolución de series, penales y estadísticas en componentes con responsabilidades claras. Evitar una reestructuración masiva antes de contar con pruebas.

Se considera terminado cuando: Las reglas pueden probarse aisladamente y la interfaz no contiene lógica deportiva.

Área de trabajo: Core/Run y componentes de dominio/simulación.

C10. Ordenar Git y archivos generados

MEJORA DE PROCESO | TRANSVERSAL - HACER PRONTO

Revisar el seguimiento de target y campeonato.dat. Incorporar una política de .gitignore y retirar del índice lo que el equipo acuerde, sin borrar código ni partidas inadvertidamente.

Se considera terminado cuando: Los commits contienen fuentes y recursos intencionales, no cambios accidentales de compilación o partidas.

Área de trabajo: Repositorio y acuerdos del equipo.

Entregables y trabajo en inglés

- N14: actualizar documentación técnica, decisiones de diseño y evidencias de POO.
- N15: conservar evidencias de gestión ágil, tareas/sprints y participación de cada integrante en inglés. Mantenerlo durante el desarrollo, no reconstruirlo al final.
- N16: preparar presentación y defensa individual/grupal, incluyendo la participación en inglés solicitada. Revisar también nombres y textos de la interfaz.

Estos entregables pueden existir fuera del repositorio; su ausencia no se confirmó. Deben verificarse con el equipo.

7. Orden de ejecución y controles de cierre

Secuencia recomendada para el equipo

- Bloque 1: C1 - persistencia y fases. No comenzar otro torneo al continuar uno guardado.
- Bloque 2: C2/C3 - habilitados, cambios y cronología. Cubrirlos con pruebas.
- Bloque 3: C4/C5/C6/C7 - desempates, criterios, penales y clasificación comprobados.
- Bloque 4: N1/N2/N3 - estadios sin repetición y ABM relacional. N4 puede avanzar en paralelo tras el bloque 2.
- Bloque 5: N4 completo y cálculos de los reportes N5-N11. N12 puede desarrollarse independientemente.
- Bloque 6: C8/N13 - presentación JavaFX y reportes visibles. La integración de interfaz sigue pospuesta por decisión del usuario.
- En todos los bloques: pruebas, C9/C10 cuando convenga y N14-N16 como proceso continuo.

Lista de aceptación final

- Un torneo normal termina con 37 encuentros y un único campeón persistente.
- Los 13 encuentros eliminatorios usan sedes distintas.
- Clasificados y ganadores coinciden con el reglamento exacto del TP.
- Suspensiones, sustituciones, pateadores y minutos reflejan participantes válidos.
- Salir y continuar desde cada fase no repite ni pierde información.
- Se conservan formaciones, incidencias, tandas y criterios de resolución.
- La interfaz final permite el flujo completo y la tabla antes/después por resultado.
- Todos los reportes y los ABM funcionan y tienen pruebas.
- Fuentes, documentación, gestión en inglés y defensa están preparados.

Fuentes y alcance

Consigna: TPGrupal-2026-Campeonato Futbol (1).pdf, requisitos de páginas 3-7 y entregables/no funcionales de páginas 9-10.
Código: versión local 79baa más cambios inspeccionados del commit remoto 47e0df9. No se hizo pull, no se modificó JavaFX ni se comprobó directamente el esquema vivo de PostgreSQL.

[Referencia del commit revisado en GitHub](#)